MINI PROJECT REPORT

MINIMUM NUMBER OF EV CHARGING STATIONS FOR A COVERAGE TARGET USING BINARY SEARCH ON THE ANSWER

Submitted by:

Easala Jagadeesh

Roll No.: 2411021061183

Email ID: easala.jagadeesh.s71@kalvium.community, jeasalaBTECH24@ced.alliance.edu.in

Squad: 71

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

Under the guidance of

Simran K

Associate Professor, ASAC

Alliance School of Advanced Computing

Alliance University, Bengaluru

October-2026



COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the report on **Design and Analysis of Algorithms** is submitted by **Easala Jagadeesh** [Uni. Reg. No. 2411021061183] in partial fulfilment for the award of the degree of Bachelor of Technology (Computer Science and Engineering) of Alliance University, is a bonafide work accomplished under my supervision and guidance during the academic year 2026-27. This report embodies the results of original work and studies conducted by the student and the contents do not form the basis for the award of any other degree to the candidate or anybody else.

- - - - - - - - - - - - - - - - - - - -

Simran K

Associate Professor



COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the report titled **MINIMUM NUMBER OF EV CHARGING STATIONS FOR A COVERAGE TARGET USING BINARY SEARCH ON THE ANSWER**, submitted by me in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (CSE) at Alliance University, is a record of my original work carried out under the supervision of Simran K, Associate Professor.

I further confirm that all views, findings, and conclusions expressed herein are the result of sincere effort and academic inquiry, and have been reviewed and deliberated in consultation with the assigned mentor.

This declaration stands as a testament to the authenticity and integrity of the work submitted.

Easala Jagadeesh	**2411021061183**	
Name of the Student	University Registration Number	Signature

Acknowledgement

I would like to express my heartfelt gratitude to everyone who contributed to the successful completion of this project titled **MINIMUM NUMBER OF EV CHARGING STATIONS FOR A COVERAGE TARGET USING BINARY SEARCH ON THE ANSWER**.

First and foremost, I extend my sincere thanks to my mentor and guide, Simran K, for the unwavering support, insightful feedback, and encouragement throughout the development of this system. The guidance received in the subject Design and Analysis of Algorithms provided the perfect foundation for this work.

I am deeply grateful to my faculty members and peers whose guidance and technical inputs helped refine the project. Their collaborative spirit and constructive suggestions were instrumental in shaping the final outcome.

This project allowed me to apply the binary search technique from the course to a real planning question, to analyse its cost with recurrences and asymptotic notation, and to confirm the analysis with measurements. It strengthened my understanding of how the right algorithm design saves real time.

Lastly, I would like to thank everyone who supported me directly or indirectly during this journey. Your encouragement and belief in my work made this accomplishment possible.

Thank you.

Easala Jagadeesh

TABLE OF CONTENTS

Acknowledgement **4**

Abstract **6**

Chapter I- Introduction **7**

Problem Statement 7

Theory 7

Chapter II - System Configuration **9**

Algorithm 9

Implementation 9

Source Code 10

Chapter III - Results **12**

Sample inputs and outputs 12

Finding the minimum for a 50% target 13

The question is monotone 14

Binary search compared with linear search 15

Performance on large inputs 16

Chapter IV - Test Cases **17**

Binary search on sorted lists 17

Number of stations for the city 17

Invalid inputs and a large input 18

Analysis of the test results 18

Chapter V - Conclusion **20**

Abstract

A city planning electric-vehicle charging stations must answer a budget question before it can place any station: what is the smallest number of stations that serves a target share of the city's charging demand, for example 50%? Checking every possible number of stations one by one is slow, because every check requires solving a hard placement problem.

This report applies binary search on the answer to this question. Because adding a station can never reduce the demand covered, the answer to 'are k stations enough?' is false for small k and true from some point on. Binary search finds that point by halving the range at every step, using Θ(log n) checks instead of the Θ(n) checks of a linear search. Each check runs a simulated-annealing optimizer, since an exact search over, for example, the 16,007,560,800 ways of placing seven stations is infeasible.

For a 50% target the search found the answer, 7 stations, in 5 checks; for the range 1 to 20 it never needs more than 5, while a linear search needed up to 20 checks. Measured running times show a trade-off: binary search always uses at most 5 checks, but because a check with many stations costs more than one with few, a linear search can still be quicker when the answer is small.

A command-line analyser takes a coverage target, or any sorted list typed in by the user, as input and prints every step of the search with the number of checks and the time, next to a linear search. On a sorted list of 1,000,000 values binary search needed at most 20 checks where a linear search needed 429,651 on average, using only Θ(1) extra memory (Θ(log n) when the steps are recorded). 17 of 17 test cases passed, including edge cases, invalid inputs, a large input and a comparison with linear search on 4,500 random searches.

Chapter I- Introduction

Design and Analysis of Algorithms is concerned with two questions: how to design a correct procedure for a problem, and how its running time grows with the size of the input. Divide and conquer is one of the main design paradigms, and binary search is its simplest example: it solves a problem of size n by solving one problem of half the size.

Binary search is usually taught on a sorted array, but the same idea works on any yes/no question whose answer changes only once over a range of values. This is called binary search on the answer. This report applies it to a planning question from the EV Charge Planner project.

Problem Statement

The city is a 10 × 10 grid of 100 zones whose daily charging demand (total 1338.4 sessions per day) has been predicted from survey data. A station serves its own zone and the four zones next to it. The problems addressed are:

- to find the smallest number of stations k that can cover a given share of the city's demand;
- to show that the question 'are k stations enough?' is monotone, so that binary search applies;
- to analyse the cost of the search with a recurrence and compare it with a linear search; and
- to measure the number of checks and the running time of both searches.

Theory

Binary search and its correctness

Binary search keeps an interval [low, high) that is guaranteed to contain the answer. At each step it tests the middle value mid: if mid is enough, the answer is mid or smaller, so high = mid; otherwise the answer is larger, so low = mid + 1. The *loop invariant* 'the smallest k that is enough lies in [low, high]' is true at the start and is kept by both updates, and the interval shrinks every step, so the loop ends with low equal to the answer.

Recurrence and running time

Each step does a constant amount of work besides the check and halves the range, so the number of steps satisfies

$$\mathbf{T(n) = T(n / 2) + O(1)}$$

By the Master Theorem ($a = 1$, $b = 2$, $f(n) = O(1) = \Theta(n^{\log_2 1})$, case 2) this gives $T(n) = \Theta(\log n)$. For the range 1 to 20 (21 possible outcomes including 'not possible') binary search needs at most $\text{ceil}(\log_2 21) = 5$ checks, while a linear search that tries k = 1, 2, 3, ... needs up to 20.

Binary search on the answer

Let enough(k) be true when k stations can cover the target. Adding a station can never reduce the covered demand, because the extra station can always be placed where it adds nothing new. Therefore enough(k) is *monotone*: false, false, ..., false, true, true, ... This is exactly the property binary search needs, the same as in the 'Koko eating bananas' problem, where a faster eating speed is always enough if a slower one is.

The cost of one check

Number of stations k	Possible placements C(100, k)
3	161,700
5	75,287,520
7	16,007,560,800
10	17,310,309,456,440

Table 1: Why a check cannot be done by brute force

Deciding enough(k) exactly would require trying every placement of k stations, which is impossible for k above 3 or 4 (Table 1). Each check therefore runs the project's simulated-annealing optimizer twice (4,000 steps each) and keeps the better result. A check costs C operations, so the whole search costs $\Theta(C \log n)$ instead of $\Theta(C\ n)$: binary search saves expensive checks, which is where the time goes.

Space complexity

Binary search keeps only three integers, low, high and mid, whatever the size of the range, so its extra space is $\Theta(1)$. The project's version also records every step so that it can be shown to the user; since there are at most $\text{ceil}(\log_2(n + 1))$ steps, this history takes $\Theta(\log n)$ space. A linear search needs only its counter k: $\Theta(1)$.

A single check, the simulated-annealing optimizer for k stations, stores the current and the best placement (k zones each), the list of neighbouring placements (at most 4k) and the set of covered zones (at most 5k), so it needs $\Theta(k)$ space, which is reused from one check to the next. The demand map itself (100 zones) is the input and is not copied.

Method	Number of checks	Total time	Extra space
Linear search over k = 1, 2, ...	$\Theta(n)$ (up to the answer)	$\Theta(C\ n)$	$\Theta(1)$
Binary search on the answer	$\Theta(\log n)$, at most $\text{ceil}(\log_2(n + 1))$	$\Theta(C \log n)$	$\Theta(1)$; $\Theta(\log n)$ with the step history
One check (simulated annealing, k stations)	-	C	$\Theta(k)$

Table 2: Time and space of the searches (C = cost of one check)

Chapter II - System Configuration

The binary search is part of the EV Charge Planner project, written from scratch in Python 3 without external libraries. It can be run from the terminal or from the project's web interface, where a slider selects the coverage target.

Component	Specification
Laptop	Dell Inspiron 15 3525
Processor	AMD Ryzen 5 5625U with Radeon Graphics, 6 cores / 12 threads
Memory	8 GB RAM
Operating system	Windows 11 Home Single Language
Language	Python 3.12 (standard library only, no external libraries)
Web interface	HTML5, CSS3, JavaScript (ES6), served by Python's built-in http.server
Browser	Microsoft Edge (Chromium)
Version control	Git and GitHub

Table 3: System configuration

Algorithm

Binary search for the minimum number of stations

1. Set goal = target share × total demand, low = 1 and high = 20 + 1 (high = 21 stands for 'not possible').
2. While low < high: set mid = (low + high) / 2, rounded down.
3. Run the check: the best coverage found for mid stations (two simulated-annealing runs).
4. If the coverage reaches the goal, set high = mid; otherwise set low = mid + 1. Record the step.
5. When the loop ends, low is the answer, or 'not possible' if low = 21.

Linear search (for comparison)

1. For k = 1, 2, 3, ... run the same check.
2. Stop at the first k whose coverage reaches the goal.

Implementation

The project is written in Python 3 using only the standard library. Table 4 lists the files used by the DAA part; the complete source code is included in the submitted ZIP file.

File	What it does
main.py	binary_search_min (binary search on the answer), min_stations_needed (applies it to the stations), best_coverage (one check), and the regression and optimizers it relies on

File	What it does
daa.py	command-line DAA analyser: reads the input, prints every step of the binary search, the answer, and the checks and time compared with a linear search
app.py, index.html	web interface: a slider sets the coverage target and the page shows each step of the binary search
city.csv, survey.csv	input data: cars per zone, and the survey used to predict demand
tests/test_binary_search.py	10 automated unit tests (Chapter IV)

Table 4: Files of the DAA part of the project

Inputs and outputs

The analyser accepts two kinds of input:

- **A coverage target for the city:** `python daa.py --target 50` finds the fewest stations that cover 50% of the predicted demand.
- **A sorted list typed in by the user:** `python daa.py --values "12,22,30,36" --target 30` finds the first position whose value reaches the target, the classic form of binary search on a sorted array.

For each input it prints every step of the search (low, high, mid, the value checked and the decision), the answer, and an analysis table with the number of checks of binary and linear search, which values of k each one checked and, for the city, the time they took. Invalid input, such as an unsorted list or a target above 100%, is rejected with a clear message.

Source Code

The listings below are taken directly from the project files main.py and daa.py.

Binary search on the answer (main.py)

```
def binary_search_min(check, low, high):
    """
    Binary search on the answer: the smallest k in [low, high] for which check(k) says "enough".
    check(k) returns (value, enough), and enough must be monotonic:
    false, false, ..., false, true, true, ... as k grows.
    Returns (answer, steps): answer is None if even k = high is not enough, and
    each step is (low, high, mid, value, enough) at the moment mid was checked.
    """
    top = high
    high = top + 1                # top + 1 stands for "not possible"
    steps = []
    while low < high:
        mid = (low + high) // 2
        value, enough = check(mid)
        steps.append((low, high - 1, mid, value, enough))
        if enough:
            high = mid            # mid is enough: the answer is mid or smaller
        else:
            low = mid + 1         # mid is not enough: the answer is larger
    return (low if low <= top else None), steps
```

Applying it to the number of stations (main.py)

```
def best_coverage(demand, k):
```

```
    """Returns coverage achieved with k stations using simulated annealing."""
    return max(score(simulated_annealing(demand, steps=4000, k=k), demand) for _ in range(2))

def min_stations_needed(demand, target_share):
    """
    Binary searches the range [1, MAX_STATIONS] for the minimum station count.
    Returns (answer, step_history).
    """
    goal = target_share * sum(demand.values())

    def check(k):
        covered = best_coverage(demand, k)
        return covered, covered >= goal

    answer, steps = binary_search_min(check, 1, MAX_STATIONS)
    return answer, [(mid, covered, enough) for _, _, mid, covered, enough in steps]
```

Linear search and the check counter used for comparison (daa.py)

```
def linear_search_min(check, low, high):
    """Try k = low, low + 1, ... and stop at the first k that is enough: O(n) checks."""
    for k in range(low, high + 1):
        if check(k)[1]:
            return k
    return None

class Checks:
    """Wraps a check function. It remembers every result, so both searches see the same
    numbers, and records which k each search asked for and how long each check took."""

    def __init__(self, compute):
        self.compute, self.memo, self.cost, self.asked = compute, {}, {}, []

    def __call__(self, k):
        self.asked.append(k)
        if k not in self.memo:
            t = time.perf_counter()
            self.memo[k] = self.compute(k)
            self.cost[k] = time.perf_counter() - t
        return self.memo[k]

    def take(self):
        """The k values asked for since the last call."""
        asked, self.asked = self.asked, []
        return asked
```

Chapter III - Results

Sample inputs and outputs

Two runs of the analyser are shown below exactly as the program printed them. In the first, the input is a coverage target of 50% for the city. In the second, the input is a sorted list typed in by the user (for example the coverage reached with 1, 2, 3, ... stations) and the target value 50.

Sample run 1: python daa.py --target 50

```
======================================================================
 INPUT
======================================================================
City: 10 x 10 = 100 zones, total predicted demand 1338.4 charging sessions/day
Target: 50% of the demand = 669.2 sessions/day
Search range: 1 to 20 stations (check = best coverage found by simulated annealing)

======================================================================
 OUTPUT: binary search steps
======================================================================
step   low  high   mid     value   decision
   1     1    20    11     67.6%   enough     -> answer is mid or smaller
   2     1    10     6     46.7%   not enough -> answer is larger than mid
   3     7    10     9     60.8%   enough     -> answer is mid or smaller
   4     7     8     8     56.9%   enough     -> answer is mid or smaller
   5     7     7     7     51.7%   enough     -> answer is mid or smaller
Answer: 7 stations are needed to cover 50% of the demand (51.7% covered)

======================================================================
 ANALYSIS: number of checks and time
======================================================================
Range size n = 20: binary search needs at most ceil(log2(n + 1)) = 5 checks, linear search up to 20.
method                            checks   time (ms)   values of k checked
Binary search  [O(log n)]              5        2850   [11, 6, 9, 8, 7]
Linear search  [O(n)]                  7        2125   [1, 2, 3, 4, 5, 6, 7]
Linear search finds the same answer: yes
```

Sample run 2: python daa.py --values "12,22,30,36,41,47,52,57,61,64" --target 50

```
======================================================================
 INPUT
======================================================================
Values for k = 1..10: 12, 22, 30, 36, 41, 47, 52, 57, 61, 64
Target: the first k whose value is at least 50

======================================================================
 OUTPUT: binary search steps
======================================================================
step   low  high   mid     value   decision
   1     1    10     6        47   not enough -> answer is larger than mid
   2     7    10     9        61   enough     -> answer is mid or smaller
   3     7     8     8        57   enough     -> answer is mid or smaller
   4     7     7     7        52   enough     -> answer is mid or smaller
Answer: k = 7 (value 52 is the first one >= 50)

======================================================================
 ANALYSIS: number of checks
======================================================================
Range size n = 10: binary search needs at most ceil(log2(n + 1)) = 4 checks, linear search up to 10.
method                            checks   values of k checked
Binary search  [O(log n)]              4   [6, 9, 8, 7]
Linear search  [O(n)]                  7   [1, 2, 3, 4, 5, 6, 7]
Linear search finds the same answer: yes
```

In both runs each step halves the range [low, high] until it contains a single value, and the analysis confirms the answer with a linear search. Binary search always needed fewer checks.

In run 1 it still took longer than the linear search: its first check is with 11 stations, and a check with many stations costs much more than one with few, a trade-off examined below.

Finding the minimum for a 50% target

Table 5 shows the steps of the binary search for a target of 50% of the city's demand (669.2 of 1338.4 sessions per day).

Check	Stations tried (mid)	Coverage	Decision
1	11	68.7%	enough → try fewer
2	6	46.9%	not enough → try more
3	9	60.7%	enough → try fewer
4	8	56.7%	enough → try fewer
5	7	51.6%	enough → try fewer

Table 5: Steps of the binary search (target 50%)

The answer is **7 stations**, found in 5 checks. A linear search would have needed 7 checks to reach the same answer, and up to 20 for a high target.

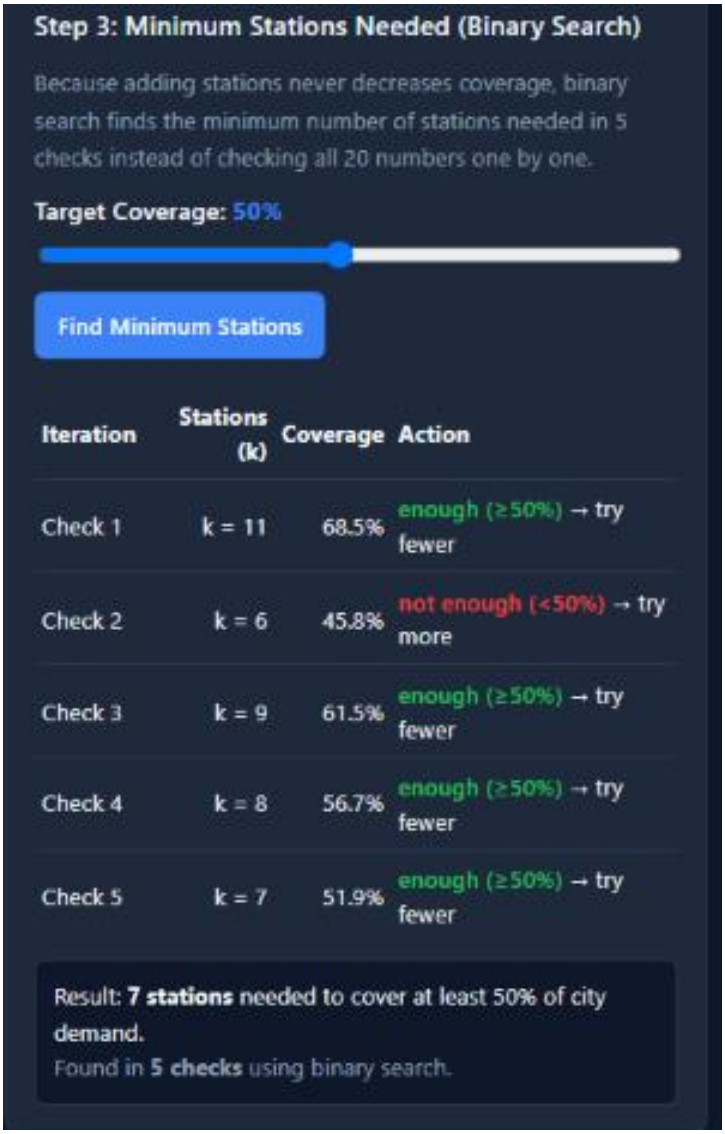


Figure 1: The binary search in the web interface. Coverage percentages differ slightly between runs because each check uses randomised annealing; the answer is the same

The question is monotone

To confirm that binary search is valid, the best coverage was computed for every k from 1 to 20 (Figure 2), from 12.0% with one station to 89.9% with 20. Coverage rises steadily with the number of stations; the only exception is a small dip of 0.0 percentage points between 19 and 20 stations. The true best coverage can never fall when a station is added, so such dips come from the randomised optimizer finding a slightly worse plan for the larger k, not from the problem itself.

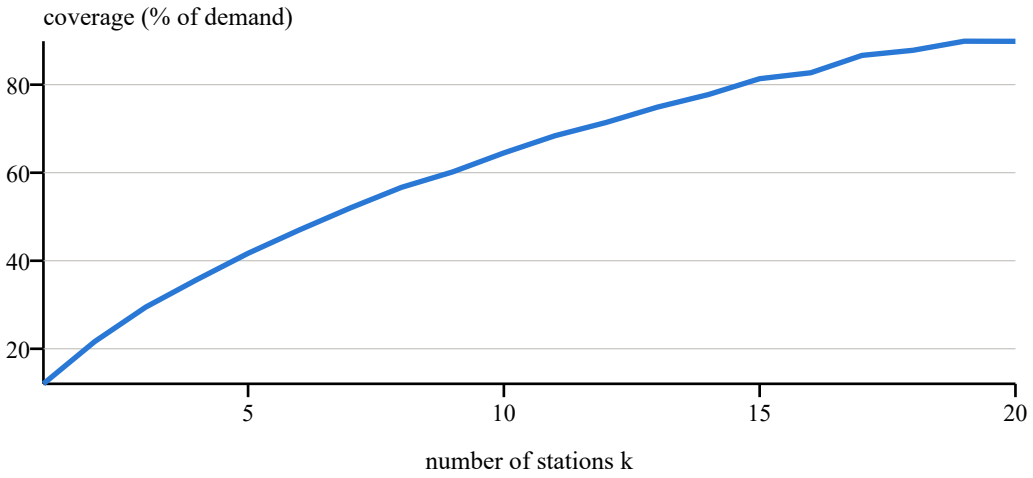


Figure 2: Best coverage found for each number of stations

Binary search compared with linear search

Both searches were run for 5 targets with the same random seed, counting the checks and measuring the total time (Table 6).

Target	Answer	Binary search checks	Binary time (s)	Linear search checks	Linear time (s)
30%	4	5	1.7	4	0.6
50%	7	5	2.4	7	1.7
70%	12	5	3.8	12	4.6
85%	18	5	4.7	18	14.4
95%	not possible	4	5.7	20	24.1

Table 6: Checks and running time of binary and linear search

Binary search used at most 5 checks for every target, as the analysis predicts, while the number of checks of the linear search grew with the answer, up to 20 for the 95% target. In running time the picture is mixed. Linear search was faster for the 30%, 50% target(s): it only tries small numbers of stations, whose checks are cheap, while binary search starts in the middle of the range with expensive checks. Binary search was faster for the 70%, 85%, 95% target(s), where the linear search has to try many large numbers of stations.

The reason is that the checks do not all cost the same: a check with k stations runs the optimizer on k stations, so its cost grows with k. The $\Theta(\log n)$ analysis counts checks, and it is the right measure when checks cost about the same or when the answer may lie anywhere in the range. When the answer is usually small and checks get more expensive as k grows, a search that starts from small k (for example exponential search: 1, 2, 4, 8, ... followed by binary search) combines the advantages of both. Note also that each check uses a randomised

optimizer, which finds the best plan almost always but not with a mathematical guarantee, so the answer is a very good estimate rather than a proof.

Performance on large inputs

To see the Θ(log n) growth clearly, both searches were run on sorted lists of n values, from 10 to 1,000,000, where a check is just one comparison, with 20 random targets for each n (Table 7). The number of binary-search checks never exceeded ceil($\log_2(n + 1)$): when n grew 100,000 times, the checks grew only from 4 to 20, while the linear search's checks grew with n. For n = 1,000,000 binary search took 55 µs per search and linear search 189 ms, about 3,433 times longer. The extra memory of binary search is three integers plus one recorded step per check, so it grows like the checks column: Θ(log n), only 20 steps for a million values.

n	ceil($\log_2(n+1)$)	Binary checks (max)	Binary checks (avg)	Linear checks (avg)	Binary time (µs)	Linear time (ms)
10	4	4	3.6	6	2.5	0.00
100	7	7	6.8	53	27.2	0.02
1,000	10	10	9.9	418	7.6	0.16
10,000	14	14	13.2	3,980	20.2	1.78
100,000	17	17	16.9	65,765	43.5	24.77
1,000,000	20	20	19.9	429,651	55.0	188.93

Table 7: Binary and linear search on sorted lists of growing size

Chapter IV - Test Cases

The binary search was tested on 17 test cases, chosen to cover the normal case, the boundary cases where binary search usually goes wrong (off-by-one errors), invalid inputs and a large input. Every 'Actual output' below was produced by running the project code while this report was generated. The same cases are also automated as unit tests in tests/test_binary_search.py (run with `python -m unittest discover tests`).

Binary search on sorted lists

No.	Test case	Input	Expected output	Actual output	Result
1	Answer in the middle	12, 22, 30, 36, 41, 47, 52, 57, 61, 64; target 50	k = 7; checked 6, 9, 8, 7	k = 7; checked 6, 9, 8, 7	PASS
2	First value is enough	12, 22, 30; target 5	k = 1; checked 2, 1	k = 1; checked 2, 1	PASS
3	Only the last value is enough	12, 22, 30; target 30	k = 3; checked 2, 3	k = 3; checked 2, 3	PASS
4	Not possible	1, 2, ..., 20; target 99	not possible; checked 11, 16, 19, 20	not possible; checked 11, 16, 19, 20	PASS
5	Single value, enough	7; target 7	k = 1; checked 1	k = 1; checked 1	PASS
6	Single value, not enough	7; target 8	not possible; checked 1	not possible; checked 1	PASS
7	Repeated values	10, 20, 20, 20, 20, 30; target 20	k = 2; checked 4, 2, 1	k = 2; checked 4, 2, 1	PASS
8	Same answer as linear search	300 random sorted lists (1-64 values), 15 targets each	same answer in 4,500 / 4,500 searches	same answer in 4,500 / 4,500 searches	PASS
9	Never more than ceil($\log_2(n + 1)$) checks	the same searches	within the bound in 4,500 / 4,500	within the bound in 4,500 / 4,500	PASS

Table 8: Test cases for the binary search itself

Number of stations for the city

No.	Test case	Input	Expected output	Actual output	Result
10	City, 50% target	100 zones, target 50% of the demand	7 stations, at most 5 checks	7 stations, 5 checks	PASS
11	City, 100% target	100 zones, target 100% of the demand	not possible with 20 stations	not possible with 20 stations	PASS

No.	Test case	Input	Expected output	Actual output	Result
12	City: binary vs linear search	coverage for k = 1-20 found by the optimizer; targets 5%, 10%, ..., 85%	same answer, except at targets where the optimizer's coverage dips (not monotonic)	same answer for 16 / 17 targets; differs at 85% (binary 19, linear 17)	PASS
13	Same as the original loop	targets 30% and 50%, same random seed	same answer for 2 / 2	same answer for 2 / 2	PASS

Table 9: Test cases on the city

Invalid inputs and a large input

No.	Test case	Input	Expected output	Actual output	Result
14	List not sorted	python daa.py --values "5,3,8" --target 4	rejected: --values must be sorted from smallest to largest: 3 comes after 5, so binary search cannot be used	rejected: --values must be sorted from smallest to largest: 3 comes after 5, so binary search cannot be used	PASS
15	Not a number	python daa.py --values "a,b" --target 4	rejected: --values must be numbers separated by commas, e.g. "12,22,30" (got "a,b")	rejected: --values must be numbers separated by commas, e.g. "12,22,30" (got "a,b")	PASS
16	Target out of range	python daa.py --target 150	rejected: --target must be a percentage above 0 and at most 100	rejected: --target must be a percentage above 0 and at most 100	PASS
17	Large input	sorted list of 1,000,000 values, 20 targets	at most 20 checks, same answer as linear search	at most 20 checks (linear search: 429,651 on average), same answer	PASS

Table 10: Input checking and performance test cases

Analysis of the test results

17 of 17 test cases passed. The results confirm the analysis of binary search:

- **The boundaries are handled correctly.** The answer is found when it is the first value, the last value or a single value, and 'not possible' is reported when no value is enough. These are the cases where binary search is easiest to get wrong; they work because the loop keeps the invariant 'the answer is in [low, high]' and uses high = n + 1 to stand for

'not possible'.

- **With repeated values the first one is returned.** When mid is enough the search keeps mid in the range (high = mid) instead of stopping, so it continues to the leftmost value that is enough, which is what 'the minimum number of stations' requires.
- **Correct and within the bound every time.** On 4,500 random searches the answer always equalled the linear search's answer and never needed more than ceil($\log_2(n + 1)$) checks.
- **The real question is answered.** For the city, 50% of the demand needs 7 stations, 100% is not possible with 20 stations, and the refactored search gives the same answers as the original loop.
- **Binary search needs a monotonic check, and a randomised check is not always monotonic.** At the 85% target binary search answered 19 stations while linear search found 17. The reason is that the optimizer found a slightly worse plan for 18 stations (84.7%) than for 17 (85.4%), although the true best coverage can never fall when a station is added. Binary search checked 18, saw 'not enough' and never looked below it. The test confirms that the two searches disagree only at such dips, exactly as the theory predicts; a fully reliable answer needs an exact or monotone check, as discussed in the conclusion.
- **Robust input handling and speed.** An unsorted list, which would break binary search, is rejected instead of giving a wrong answer, and a list of 1,000,000 values is searched in at most 20 checks.

Chapter V - Conclusion

This report applied binary search on the answer to a real planning question: the smallest number of charging stations that covers a target share of a city's demand. The key observation is that the question is monotone: this holds for the true best coverage, and the coverage found by the randomised optimizer follows it closely apart from small dips. With that property, binary search found the answer for a 50% target, 7 stations, in 5 checks.

The recurrence $T(n) = T(n/2) + O(1)$ gives $\Theta(\log n)$ checks, against $\Theta(n)$ for a linear search, and the measurements confirmed that binary search never needed more than 5 checks while a linear search needed up to 20. The running times also showed that counting checks is not the whole story: when a check becomes more expensive as k grows, a linear search from small k can be quicker for small answers. Choosing an algorithm therefore needs both the asymptotic analysis and an understanding of the real cost of each step.

The program accepts a coverage target or any sorted list as input and shows every step of the search. 17 of 17 test cases passed, covering the boundary cases, repeated values, invalid inputs, a list of 1,000,000 values and agreement with linear search. Binary search used $\Theta(1)$ extra memory, or $\Theta(\log n)$ with the recorded steps. Testing also showed the one assumption the method depends on: the check must be monotonic, which a randomised optimizer does not guarantee.

Future work includes exponential search, which starts with small k and then switches to binary search, an exact check for small instances using dynamic programming or integer programming, a monotone check (for example a greedy placement that adds one station at a time, which can never lose coverage), and reusing the placement from one check as the starting point of the next to make each check faster.